

Sprint Documentation

Sprint 14	Start Date: 30/03/2026	Final Date: 03/04/2026
Projetc Name:	Civitas-backend	
Ano: 5º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Wendell Alves Nascimento		
Gustavo Henrique Moreira Porto		

Sprint Backlog

Task#	Description	Start Date	Final date
#58	Implementação de Autorização com JWT	30/03/2026	03/04/2026

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
#58		Wendell Alves Nascimento, Gustavo Henrique Moreira Porto	Completo	10 horas	10 horas

Sprint Description

Tarefa: Sprint 14 - Implementação de Autorização com JWT

Objetivo

Implementar a **autorização baseada em JWT**, protegendo as rotas da API e garantindo que apenas usuários autenticados possam acessar os recursos do sistema.

Critérios de Aceite

- Configurar autenticação JWT na aplicação.

Validar automaticamente o token enviado nas requisições.

Proteger rotas da API utilizando autorização.

Bloquear acesso a endpoints protegidos quando não houver token.

Bloquear acesso quando o token for inválido ou expirado.

Permitir acesso quando o token for válido.

Atualizar documentação da API.

Escopo da Implementação

A autenticação deverá funcionar através do header:

Authorization: Bearer {token}

Rotas protegidas devem exigir autenticação válida.

Observações

- O token deve ser validado automaticamente pelo middleware da aplicação.
- Endpoints protegidos devem utilizar mecanismo de autorização (exemplo: [Authorize]).
- Endpoints públicos (como login) não devem exigir autenticação.
- A API deve retornar **401 Unauthorized** quando o token estiver ausente ou inválido.
- A rota Login deve permitir requisições mesmo sem autorização.
- Permitir a geração de tokens personalizados
- No appsettings.json deve ter uma propriedade chamada DEV, do tipo Boolean onde TRUE desativa a necessidade de token nas rotas e FALSE ativa a necessidade do token nas rotas.

Sprint Results

Arquivos alterados:

appsettings.json

adicionada a propriedade "DEV": false

Program.cs

adicionados using

registrado o handler de bypass de autorização

adicionada configuração do Swagger com AddSecurityRequirement

Services/Security/DevBypassAuthorizationHandler.cs

arquivo novo

Services/Security/IJwtTokenService.cs

adicionado parâmetro extraClaims

Services/Security/JwtTokenService.cs

implementado suporte a extraClaims

12 controllers resource

adicionado using de autorização

adicionado atributo [Authorize]

Tests/AuthorizationEndpointsTests.cs

arquivo novo

adicionados 7 cenários de teste

Tests/UsuarioValidationEndpointsTests.cs

adicionado CreateAuthenticatedClient()

adicionado uso de token

Tests/PaginationEndpointsTests.cs

adicionado CreateAuthenticatedClient()

adicionado uso de token

Resumo da conclusão

A estratégia foi de menor impacto possível, sem criar novas camadas, sem middleware novo e sem policy customizada complexa.

O modo DEV foi implementado com um DevBypassAuthorizationHandler, registrado como singleton de IAuthorizationHandler. Quando DEV=true, ele libera os requisitos pendentes de autorização antes que a

validação JWT bloqueie a requisição. Assim, toda a lógica ficou centralizada e nenhum controller precisou receber if ou tratamento especial.

A proteção das rotas foi feita com a adição de [Authorize] na classe de cada controller de recurso, mantendo o login público.

Também foi adicionado suporte a claims extras no JWT com um parâmetro opcional IEnumerable<Claim>? extraClaims = null, preservando retrocompatibilidade.

Testes criados e ajustados

Novo: AuthorizationEndpointsTests.cs

login sem token retorna 200

endpoint protegido sem token retorna 401 com DEV=false

endpoint protegido com token inválido retorna 401 com DEV=false

endpoint protegido com token expirado retorna 401 com DEV=false

endpoint protegido com token válido retorna 200 com DEV=false

endpoint protegido sem token retorna 200 com DEV=true

token com claims adicionais retorna 200

Ajustados:

UsuarioValidationEndpointsTests

PaginationEndpointsTests

ambos passaram a usar CreateAuthenticatedClient() com JWT de teste

Resultado final

27/27 testes passando

Validação manual

Com DEV=false, chamar GET /api/usuarios sem token deve retornar 401

Após fazer POST /api/auth/login e enviar Authorization: Bearer {token}, a rota protegida deve retornar 200

Com DEV=true, após reiniciar a aplicação, as rotas protegidas devem funcionar sem token

No Swagger UI, os endpoints devem mostrar cadeado e aceitar Bearer {token} no botão Authorize

Token customizado pode ser gerado com:

```
JwtTokenService.GenerateToken(usuario, extraClaims: new[] { new Claim("role", "gestor") })
```

Riscos e observações finais

O 401 padrão do ASP.NET Core acontece antes do controller, então a resposta não usa o envelope customizado Response

Isso foi mantido por ser mais seguro e por estar fora do escopo criar um handler de autenticação customizado

DEV=true não deve ficar como padrão; a entrega deixou false

Os warnings de build já existiam antes e não foram introduzidos por essa sprint

Assinaturas